SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week7 – Monday**	**Batch**	B-48
NAME	K.Srija	**Hall Ticket**	2303A54023
Assignment Number: 13.1			

Q.No.	Question	*Expected Time to complete*
1	**Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions** **Lab Objectives:** • Identify code smells and inefficiencies in legacy Python	Week7 - Monday

scripts.

- Use AI-assisted coding tools to **refactor** for readability, maintainability, and performance.
- Apply **modern Python best practices** while ensuring output correctness.

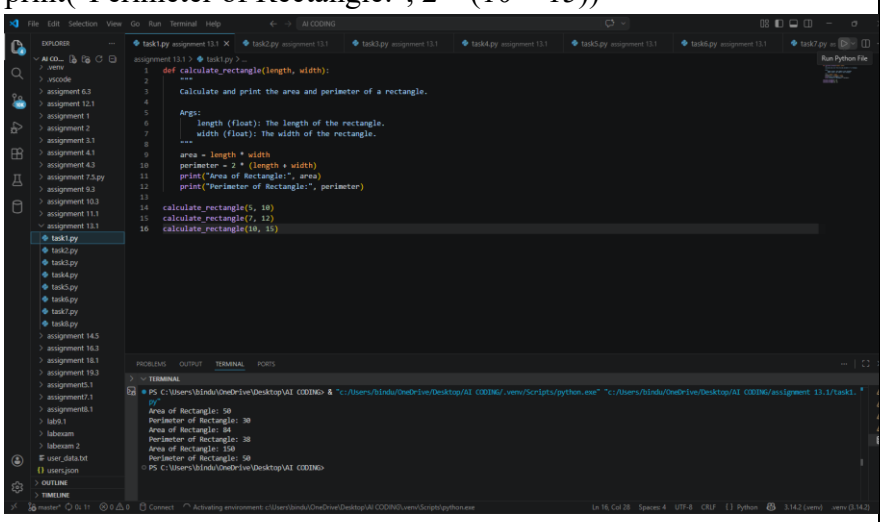
Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
  - Prompt AI to identify duplicate logic and replace it with functions or classes.
  - Ensure the refactored code maintains the same output.
  - Add docstrings to all functions.
- Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```



Explanation:

The original code repeatedly calculates the area and perimeter of rectangles using hardcoded values. This duplication makes the code longer and harder to maintain. The refactored version introduces a

reusable function that takes length and width as inputs and performs the calculations

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

- Task: Use AI to analyze a Python script with nested loops and complex conditionals.
- Instructions:
  - Ask AI to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).
  - Implement changes while keeping logic intact.
  - Compare execution time before and after refactoring.
- Sample Legacy Code:

```
# Legacy inefficient code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

task1.py assignment 13.1 | task2.py assignment 13.1 | task3.py assignment 13.1 | task4.py assignment 13.1 | task5.py assignment 13.1 | task6

assignment 13.1 > task2.py > ...

```python
#- Replace nested loops with a more efficient approach (like set lookup).
#- Maintain the same output.
#- Add execution time comparison (before and after).
#- Briefly explain the improvement.
import time
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\bindu\OneDrive\Desktop\AI CODING> & "c:/Users/bindu/OneDrive/Desktop/AI CODING/.venv/Scripts/python.exe" "c:/Users/bindu/OneDrive/De
py"
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\bindu\OneDrive\Desktop\AI CODING> & "c:\Users\bindu\OneDrive\Desktop\AI CODING\.venv\Scripts\Activate.ps1"
(.venv) PS C:\Users\bindu\OneDrive\Desktop\AI CODING> & "c:/Users/bindu/OneDrive/Desktop/AI CODING/.venv/Scripts/python.exe" "c:/Users/bindu/One
1/task2.py"
Charlie is in the list
Eve is not in the list
Bob is in the list
(.venv) PS C:\Users\bindu\OneDrive\Desktop\AI CODING>
```

Explanation:

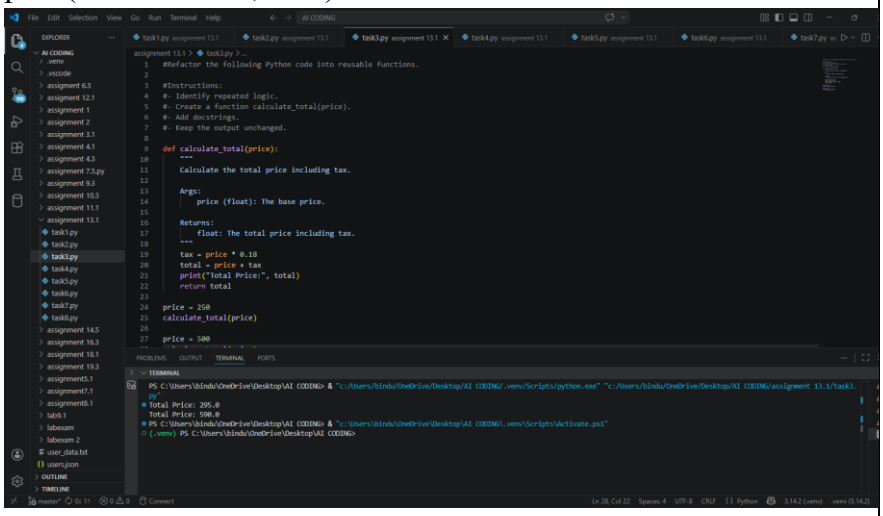
The original code uses nested loops to check if elements exist in a list, which increases time complexity. The optimized version converts the list into a set, allowing faster lookups. This reduces time complexity and improves performance. The logic remains the same, but execution becomes more efficient.

Task Description #3 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:
  - Identify repeated or related logic and extract it into reusable functions.
  - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
```

tax = price * 0.18
total = price + tax
print("Total Price:", total)



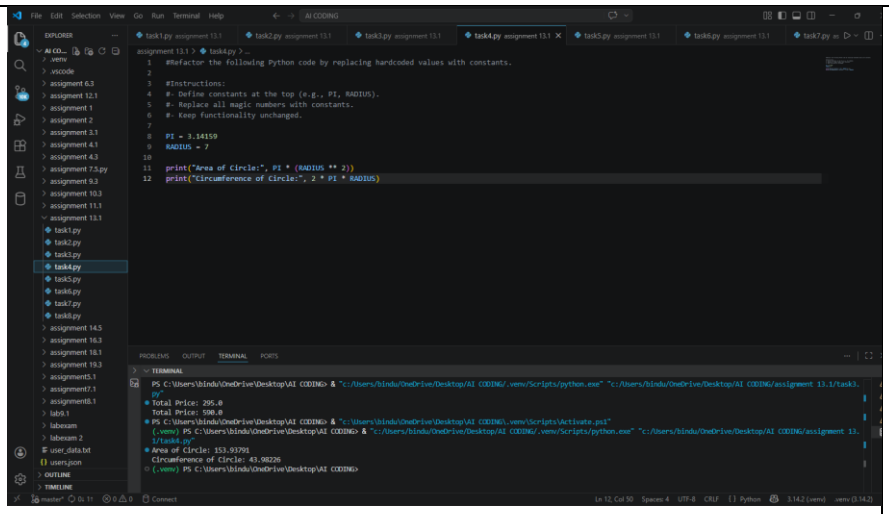
Explanation

The legacy code repeats tax and total price calculations for different values. This repetition is inefficient and not scalable. The refactored version extracts the logic into a function calculate_total(price), which can be reused for any price. This makes the code modular, cleaner, and easier to update.

Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

- Task: Use AI to identify and replace all hardcoded "magic numbers" in the code with named constants.
- Instructions:
  - Create constants at the top of the file.
  - Replace all hardcoded occurrences in calculations with these constants.
  - Ensure the code remains functional and is easier to maintain.
- Sample Legacy Code:

# Legacy script with hardcoded values
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)

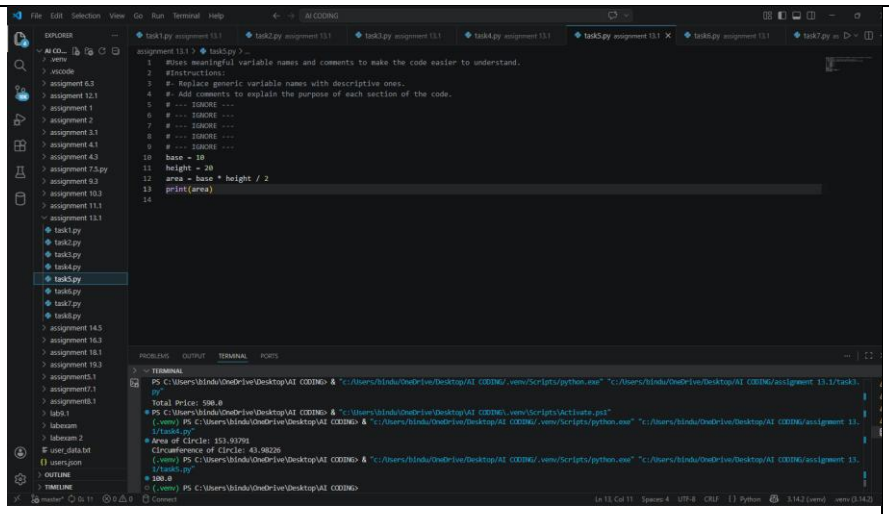


Explanation:
The original code uses fixed numeric values (magic numbers) like 3.14159 and 7 directly in calculations. This reduces readability and flexibility. The refactored version replaces these with named constants (PI and RADIUS), making the code easier to understand and maintain.

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

- Task: Use AI to improve readability by renaming unclear variables and adding inline comments.
- Instructions:
  - Replace vague names with meaningful ones.
  - Add comments where logic is not obvious.
  - Keep functionality exactly the same.
- Sample Legacy Code:

```
# Legacy script with poor variable names
a = 10
b = 20
c = a * b / 2
print(c)
```



Explanation

The original code uses unclear variable names like a, b, and c, making it difficult to understand the logic. The improved version uses meaningful names such as base, height, and area_of_triangle, along with comments. This enhances readability and makes the code self-explanatory.

Task Description #6 (Refactoring – Removing Redundant Conditional Logic)

• Task: Use AI to refactor a Python script that contains repeated if–else logic for grading students.

• Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function.

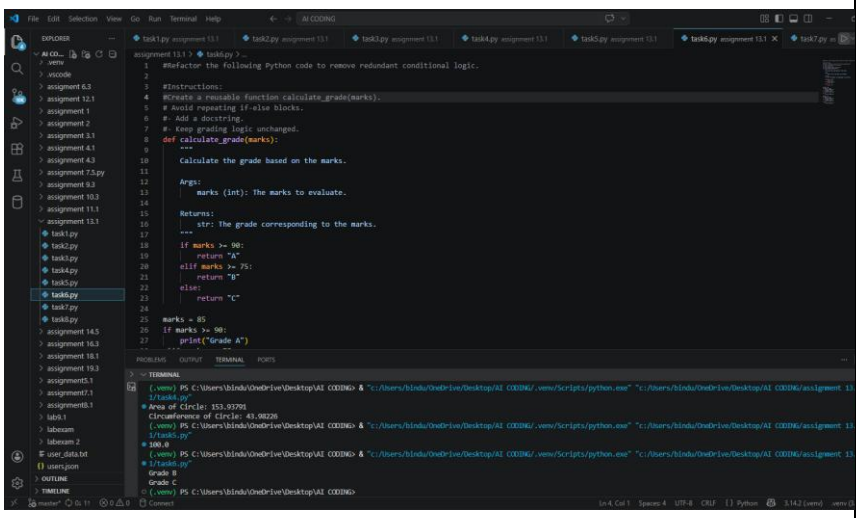
o Ensure grading logic remains unchanged.

• Code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
```

```
    print("Grade C")


marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```



Explanation

The same grading logic is repeated multiple times in the original code.This leads to redundancy and poor maintainability. The refactored version creates a function calculate_grade(marks) that handles grading logic in one place. This reduces duplication and makes future updates easier.

Task Description #7 (Refactoring – Converting Procedural Code to Functions)

- **Task: Use AI to refactor procedural input–processing logic into functions.**
- Instructions:

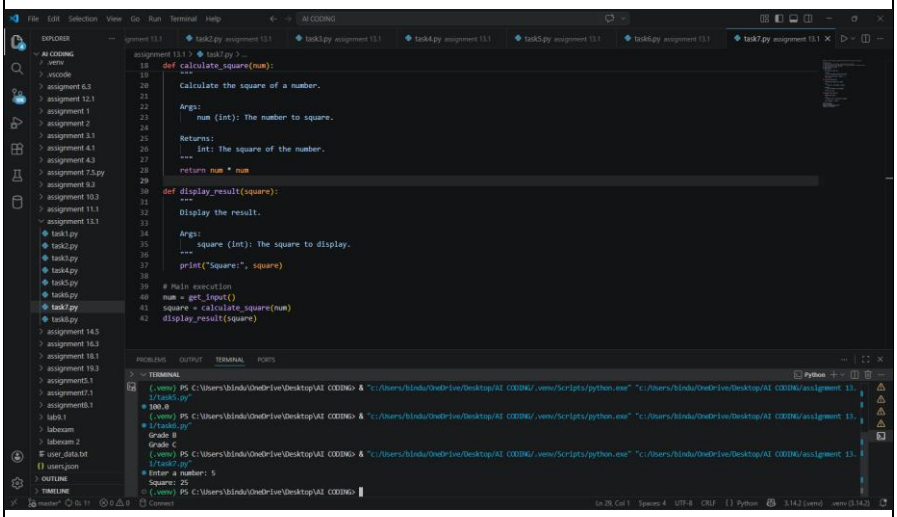
o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

```
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```



Explanation:

The original code performs input, processing, and output in a single block, making it less structured. The refactored version separates these into functions (get_input, calculate_square, display_result). This modular approach improves readability, testing, and reusability.

Task Description #8 (Refactoring – Optimizing List Processing)

• Task: Use AI to refactor inefficient list processing logic.

• Instructions:

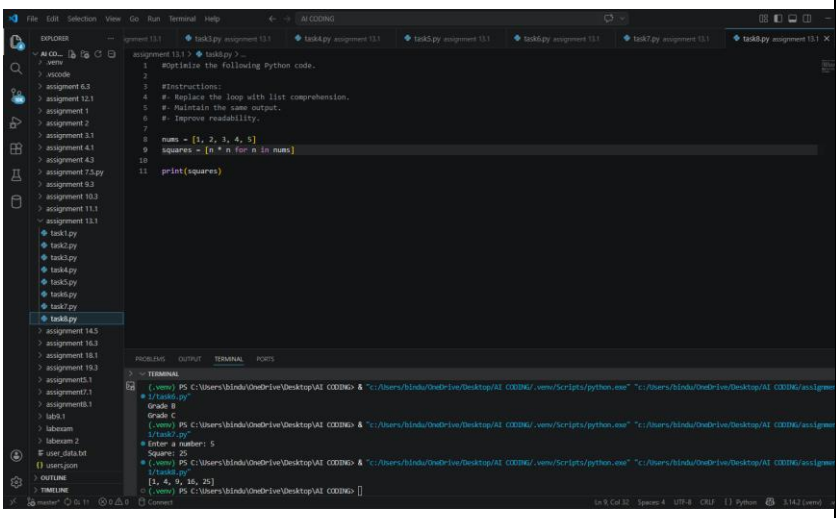
o Ask AI to replace manual loops with list comprehensions or built-in functions.

o Ensure output remains identical.

• Sample Legacy Code:

```
nums = [1, 2, 3, 4, 5]
squares = []
for n in nums:
    squares.append(n * n)
```

print(squares)



Explanation:

The original code uses a loop to generate squares of numbers and append them to a list. This can be simplified using list comprehension. The optimized version uses a single line to achieve the same result, making the code shorter, cleaner, and more Pythonic.